

Handle background task cancellation

In-process background tasks are cancelled the same way as out-of-process background tasks are (see [Handle a cancelled background task](#)). Be aware that your **BackgroundActivated** event handler must exit before the cancellation occurs, or the whole process will be terminated. If your foreground app closes unexpectedly when you cancel the background task, verify that your handler exited before the cancellation occurred.

The manifest

Unlike out-of-process background tasks, you are not required to add background task information to the package manifest in order to run in-process background tasks.

Summary and next steps

You should now understand the basics of how to write an in-process background task.

See the following related topics for API reference, background task conceptual guidance, and more detailed instructions for writing apps that use background tasks.

Related topics

Detailed background task instructional topics

- [Convert an out-of-process background task to an in-process background task](#)
- [Create and register an out-of-process background task](#)
- [Play media in the background](#)
- [Respond to system events with background tasks](#)
- [Register a background task](#)
- [Set conditions for running a background task](#)
- [Use a maintenance trigger](#)
- [Handle a cancelled background task](#)
- [Monitor background task progress and completion](#)
- [Run a background task on a timer](#)

Background task guidance

- [Guidelines for background tasks](#)
- [Debug a background task](#)

- [How to trigger suspend, resume, and background events in UWP apps \(when debugging\)](#)

Background Task API Reference

- [Windows.ApplicationModel.Background](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Create and register an out-of-process background task

Article • 10/20/2022

Important APIs

- [IBackgroundTask](#)
- [BackgroundTaskBuilder](#)
- [BackgroundTaskCompletedEventHandler](#)

Create a background task class and register it to run when your app is not in the foreground. This topic demonstrates how to create and register a background task that runs in a separate process than your app's process. To do background work directly in the foreground application, see [Create and register an in-process background task](#).

ⓘ Note

If you use a background task to play media in the background, see [Play media in the background](#) for information about improvements in Windows 10, version 1607, that make it much easier.

ⓘ Note

If you are implementing an out-of-process background task in a C# desktop application with .NET 6 or later, then use the [C#/WinRT authoring support](#) to create a Windows Runtime Component. This applies for apps using the Windows App SDK, WinUI 3, WPF, or WinForms. See the [Background task sample](#) [↗](#) for an example.

Create the Background Task class

You can run code in the background by writing classes that implement the [IBackgroundTask](#) interface. This code runs when a specific event is triggered by using, for example, [SystemTrigger](#) or [MaintenanceTrigger](#).

The following steps show you how to write a new class that implements the [IBackgroundTask](#) interface.

1. Create a new project for background tasks and add it to your solution. To do this, right-click on your solution node in the **Solution Explorer** and select **Add > New Project**. Then select the **Windows Runtime Component** project type, name the project, and click OK.
2. Reference the background tasks project from your Universal Windows Platform (UWP) app project. For a C# or C++ app, in your app project, right-click on **References** and select **Add New Reference**. Under **Solution**, select **Projects** and then select the name of your background task project and click **Ok**.
3. To the background tasks project, add a new class that implements the **IBackgroundTask** interface. The **IBackgroundTask.Run** method is a required entry point that will be called when the specified event is triggered; this method is required in every background task.

ⓘ Note

The background task class itself—and all other classes in the background task project—need to be **public** classes that are **sealed** (or **final**).

The following sample code shows a very basic starting point for a background task class.

C#

```
// ExampleBackgroundTask.cs
using Windows.ApplicationModel.Background;

namespace Tasks
{
    public sealed class ExampleBackgroundTask : IBackgroundTask
    {
        public void Run(IBackgroundTaskInstance taskInstance)
        {
        }
    }
}
```

4. If you run any asynchronous code in your background task, then your background task needs to use a deferral. If you don't use a deferral, then the background task process can terminate unexpectedly if the **Run** method returns before any asynchronous work has run to completion.

Request the deferral in the **Run** method before calling the asynchronous method. Save the deferral to a class data member so that it can be accessed from the asynchronous method. Declare the deferral complete after the asynchronous code completes.

The following sample code gets the deferral, saves it, and releases it when the asynchronous code is complete.

C#

```
BackgroundTaskDeferral _deferral; // Note: defined at class scope so that we
can mark it complete inside the OnCancel() callback if we choose to support
cancellation
public async void Run(IBackgroundTaskInstance taskInstance)
{
    _deferral = taskInstance.GetDeferral();
    //
    // TODO: Insert code to start one or more asynchronous methods using the
    //       await keyword, for example:
    //
    // await ExampleMethodAsync();
    //

    _deferral.Complete();
}
```

ⓘ Note

In C#, your background task's asynchronous methods can be called using the **async/await** keywords. In C++/CX, a similar result can be achieved by using a task chain.

For more information about asynchronous patterns, see [Asynchronous programming](#). For additional examples of how to use deferrals to keep a background task from stopping early, see the [background task sample](#) [↗](#).

The following steps are completed in one of your app classes (for example, MainPage.xaml.cs).

ⓘ Note

You can also create a function dedicated to registering background tasks—see [Register a background task](#). In that case, instead of using the next three steps, you can simply construct the trigger and provide it to the registration function along with the task name, task entry point, and (optionally) a condition.

Register the background task to run

1. Find out whether the background task is already registered by iterating through the [BackgroundTaskRegistration.AllTasks](#) property. This step is important; if your app doesn't check for existing background task registrations, it could easily register the task multiple times, causing issues with performance and maxing out the task's available CPU time before work can complete.

The following example iterates on the **AllTasks** property and sets a flag variable to true if the task is already registered.

C#

```
var taskRegistered = false;
var exampleTaskName = "ExampleBackgroundTask";

foreach (var task in BackgroundTaskRegistration.AllTasks)
{
    if (task.Value.Name == exampleTaskName)
    {
        taskRegistered = true;
        break;
    }
}
```

2. If the background task is not already registered, use [BackgroundTaskBuilder](#) to create an instance of your background task. The task entry point should be the name of your background task class prefixed by the namespace.

The background task trigger controls when the background task will run. For a list of possible triggers, see [SystemTrigger](#).

For example, this code creates a new background task and sets it to run when the **TimeZoneChanged** trigger occurs:

C#

```
var builder = new BackgroundTaskBuilder();

builder.Name = exampleTaskName;
builder.TaskEntryPoint = "Tasks.ExampleBackgroundTask";
builder.SetTrigger(new SystemTrigger(SystemTriggerType.TimeZoneChange,
false));
```

3. You can add a condition to control when your task will run after the trigger event occurs (optional). For example, if you don't want the task to run until the user is present, use the condition **UserPresent**. For a list of possible conditions, see [SystemConditionType](#).

The following sample code assigns a condition requiring the user to be present:

C#

```
builder.AddCondition(new SystemCondition(SystemConditionType.UserPresent));
```

4. Register the background task by calling the `Register` method on the [BackgroundTaskBuilder](#) object. Store the [BackgroundTaskRegistration](#) result so it can be used in the next step.

The following code registers the background task and stores the result:

C#

```
BackgroundTaskRegistration task = builder.Register();
```

ⓘ Note

Universal Windows apps must call [RequestAccessAsync](#) before registering any of the background trigger types.

To ensure that your Universal Windows app continues to run properly after you release an update, use the **ServicingComplete** (see [SystemTriggerType](#)) trigger to perform any post-update configuration changes such as migrating the app's database and registering background tasks. It is best practice to unregister background tasks associated with the previous version of the app (see [RemoveAccess](#)) and register background tasks for the new version of the app (see [RequestAccessAsync](#)) at this time.

For more information, see [Guidelines for background tasks](#).

Handle background task completion using event handlers

You should register a method with the [BackgroundTaskCompletedEventHandler](#), so that your app can get results from the background task. When the app is launched or resumed, the marked method will be called if the background task has completed since the last time the app was in the foreground. (The `OnCompleted` method will be called immediately if the background task completes while your app is currently in the foreground.)

1. Write an `OnCompleted` method to handle the completion of background tasks. For example, the background task result might cause a UI update. The method footprint shown here is required for the `OnCompleted` event handler method, even though this example does not use the *args* parameter.

The following sample code recognizes background task completion and calls an example UI update method that takes a message string.

C#

```
private void OnCompleted(IBackgroundTaskRegistration task,
BackgroundTaskCompletedEventArgs args)
{
    var settings = Windows.Storage.ApplicationData.Current.LocalSettings;
    var key = task.TaskId.ToString();
    var message = settings.Values[key].ToString();
    UpdateUI(message);
}
```

ⓘ Note

UI updates should be performed asynchronously, to avoid holding up the UI thread. For an example, see the `UpdateUI` method in the [background task sample](#) [↗].

2. Go back to where you registered the background task. After that line of code, add a new `BackgroundTaskCompletedEventHandler` object. Provide your `OnCompleted` method as the parameter for the `BackgroundTaskCompletedEventHandler` constructor.

The following sample code adds a `BackgroundTaskCompletedEventHandler` to the `BackgroundTaskRegistration`:

C#

```
task.Completed += new BackgroundTaskCompletedEventHandler(OnCompleted);
```

Declare in the app manifest that your app uses background tasks

Before your app can run background tasks, you must declare each background task in the app manifest. If your app attempts to register a background task with a trigger that

isn't listed in the manifest, the registration of the background task will fail with a "runtime class not registered" error.

1. Open the package manifest designer by opening the file named `Package.appxmanifest`.
2. Open the **Declarations** tab.
3. From the **Available Declarations** drop-down, select **Background Tasks** and click **Add**.
4. Select the **System event** checkbox.
5. In the **Entry point:** textbox, enter the namespace and name of your background class which is for this example is `Tasks.ExampleBackgroundTask`.
6. Close the manifest designer.

The following Extensions element is added to your `Package.appxmanifest` file to register the background task:

XML

```
<Extensions>
  <Extension Category="windows.backgroundTasks"
EntryPoint="Tasks.ExampleBackgroundTask">
    <BackgroundTasks>
      <Task Type="systemEvent" />
    </BackgroundTasks>
  </Extension>
</Extensions>
```

Summary and next steps

You should now understand the basics of how to write a background task class, how to register the background task from within your app, and how to make your app recognize when the background task is complete. You should also understand how to update the application manifest so that your app can successfully register the background task.

📌 Note

Download the [background task sample](#) to see similar code examples in the context of a complete and robust UWP app that uses background tasks.

See the following related topics for API reference, background task conceptual guidance, and more detailed instructions for writing apps that use background tasks.

Related topics

Detailed background task instructional topics

- [Respond to system events with background tasks](#)
- [Register a background task](#)
- [Set conditions for running a background task](#)
- [Use a maintenance trigger](#)
- [Handle a cancelled background task](#)
- [Monitor background task progress and completion](#)
- [Run a background task on a timer](#)
- [Create and register an in-process background task.](#)
- [Convert an out-of-process background task to an in-process background task](#)

Background task guidance

- [Guidelines for background tasks](#)
- [Debug a background task](#)
- [How to trigger suspend, resume, and background events in UWP apps \(when debugging\)](#)

Background Task API Reference

- [Windows.ApplicationModel.Background](#)

Feedback

Was this page helpful?



[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Create and register a Win32 COM background task

Article • 10/20/2022

Tip

The `BackgroundTaskBuilder.SetTaskEntryPointClsid` method is available starting in Windows 10, version 2004.

Note

This scenario is only applicable to packaged Win32 apps. UWP applications will encounter errors trying to implement this scenario.

Important APIs

- [IBackgroundTask](#)
- [BackgroundTaskBuilder](#)

Create a COM background task class and register it to run in your full trust packaged Win32 app in response to triggers. You can use background tasks to provide functionality when your app is suspended or not running. This topic demonstrates how to create and register a background task that can run in your foreground app process or another process.

Create the Background Task class

You can run code in the background by writing classes that implement the [IBackgroundTask](#) interface. This code runs when a specific event is triggered by using, for example, [SystemTrigger](#) or [TimeTrigger](#).

The following steps show you how to write a new class that implements the [IBackgroundTask](#) interface and add it to your main process.

1. [See these instructions](#) to reference WinRT APIs in your packaged Win32 application solution. This is required to use the `IBackgroundTask` and related APIs.
2. In that new class, implement the [IBackgroundTask](#) interface. The [IBackgroundTask.Run](#) method is a required entry point that will be called when the specified event is triggered; this method is required in every background task.

ⓘ Note

The background task class itself—and all other classes in the background task project—need to be **public**.

The following sample code shows a basic a background task class that counts primes and writes it to a file until it is requested to be canceled.

The C++/WinRT example implements the background task class as a **COM coclass**.

▼ Background task code sample

C#

```
using System;
using System.IO; // Path
using System.Threading; // EventWaitHandle
using System.Collections.Generic; // Queue
using System.Runtime.InteropServices; // Guid, RegistrationServices
using Windows.ApplicationModel.Background; // IBackgroundTask

namespace PackagedWinMainBackgroundTaskSample
{
    // {14C5882B-35D3-41BE-86B2-5106269B97E6} is GUID to register this task
    // with BackgroundTaskBuilder. Generate a random GUID before implementing.
    [ComVisible(true)]
    [ClassInterface(ClassInterfaceType.None)]
    [Guid("14C5882B-35D3-41BE-86B2-5106269B97E6")]
    [ComSourceInterfaces(typeof(IBackgroundTask))]
    public class SampleTask : IBackgroundTask
    {
        private volatile int cleanupTask; // flag used to indicate to Run
        // method that it should exit
        private Queue<int> numbersQueue; // the data structure holding the
        // set of primes in memory

        private const int maxPrimeNumber = 1000000000; // the number up to
        // which task will attempt to calculate primes
        private const int queueDepthToWrite = 10; // how frequently this
        // task should flush its queue of primes
        private const string numbersQueueFile = "numbersQueue.log"; // the
        // file to write to relative to AppData

        public SampleTask()
        {
            cleanupTask = 0;
            numbersQueue = new Queue<int>(queueDepthToWrite);
        }

        /// <summary>
        /// This method writes all the numbers in the current queue to the
```

specified file.

```
    /// </summary>
    private void FlushNumbersToFile(Queue<int> queueToWrite)
    {
        string logPath =
Path.Combine(ApplicationData.Current.LocalFolder.Path,

System.Diagnostics.Process.GetCurrentProcess().ProcessName);

        if (!Directory.Exists(logPath))
        {
            Directory.CreateDirectory(logPath);
        }

        logPath = Path.Combine(logPath, numbersQueueFile);

        const string delimiter = ", ";
        UnicodeEncoding unicodeEncoding = new UnicodeEncoding();
        // convert the queue to a list of comma separated values.
        string stringToWrite = String.Join(delimiter, queueToWrite);
        // Add the comma at the end.
        stringToWrite += delimiter;

        File.AppendAllText(logPath, stringToWrite);
    }

    /// <summary>
    /// This method determines if the specified number is a prime
number.
    /// </summary>
    private bool IsPrimeNumber(int dividend)
    {
        bool isPrime = true;
        for (int divisor = dividend - 1; divisor > 1; divisor -= 1)
        {
            if ((dividend % divisor) == 0)
            {
                isPrime = false;
                break;
            }
        }

        return isPrime;
    }

    /// <summary>
    /// Given the current number, this method calculates the next prime
number (excluding the specified number).
    /// </summary>
    private int GetNextPrime(int previousNumber)
    {
        int currentNumber = previousNumber + 1;
        while (!IsPrimeNumber(currentNumber))
        {
            currentNumber += 1;
        }
    }
}
```

```

    }

    return currentNumber;
}

/// <summary>
/// This method is the main entry point for the background task. The
system will believe this background task
/// is complete when this method returns.
/// </summary>
[MTAThread]
public void Run(IBackgroundTaskInstance taskInstance)
{
    // Start with the first applicable number.
    int currentNumber = 1;

    taskDeferral = taskInstance.GetDeferral();

    // Wire the cancellation handler.
    taskInstance.Canceled += this.OnCanceled;

    // Set the progress to indicate this task has started
    taskInstance.Progress = 10;

    // Calculate primes until a cancellation has been requested or
until
    // the maximum number is reached.
    while ((cleanupTask == 0) && (currentNumber < maxPrimeNumber)) {
        // Compute the next prime number and add it to our queue.
        currentNumber = GetNextPrime(currentNumber);
        numbersQueue.Enqueue(currentNumber);
        // Once the queue is filled to its max size, flush the
numbers to the file.
        if (numbersQueue.Count >= queueDepthToWrite)
        {
            FlushNumbersToFile(numbersQueue);
            numbersQueue.Clear();
        }
    }

    // Flush any remaining numbers to the file as part of cleanup.
    FlushNumbersToFile(numbersQueue);

    if (taskDeferral != null)
    {
        taskDeferral.Complete();
    }
}

/// <summary>
/// This method is signaled when the system requests the background
task be canceled. This method will signal
/// to the Run method to clean up and return.
/// </summary>
[MTAThread]

```

```

        public void OnCanceled(IBackgroundTaskInstance taskInstance,
BackgroundTaskCancellationReason cancellationReason)
        {
            cleanupTask = 1;
        }
    }
}

```

Add the support code to instantiate the COM class

In order for the background task to be activated into a full trust Win32 application, the background task class must have support code such that COM understands how to start the app process if it is not running, and then understanding which instance of the process is currently the server for handling new activations for that background task.

1. COM needs to understand how to launch the app process if it is not running already. The app process that hosts the background task code needs to be declared in the package manifest. The following sample code shows how the **SampleTask** is hosted inside **SampleBackgroundApp.exe**. When the background task is launched when no process is running, **SampleBackgroundApp.exe** will be launched with process arguments **"-StartSampleTaskServer"**.

XML

```

<Extensions>
  <com:Extension Category="windows.comServer">
    <com:ComServer>
      <com:ExeServer
Executable="SampleBackgroundApp\SampleBackgroundApp.exe"
DisplayName="SampleBackgroundApp" Arguments="-StartSampleTaskServer">
        <com:Class Id="14C5882B-35D3-41BE-86B2-5106269B97E6"
DisplayName="Sample Task" />
      </com:ExeServer>
    </com:ComServer>
  </com:Extension>
</Extensions>

```

2. Once your process is started with the right arguments, it should tell COM that it is the current COM server for new instances of **SampleTask**. The following sample code shows how the application process should register itself with COM. Note these samples indicate how the process would declare itself as the COM server for

SampleTask for at least one instance to complete before exiting. This is optional, and handling a background task may start your main process functions.

C#

```
class SampleTaskServer
{
    SampleTaskServer()
    {
        comRegistrationToken = 0;
        waitHandle = new EventWaitHandle(false, EventResetMode.AutoReset);
    }

    ~SampleTaskServer()
    {
        Stop();
    }

    public void Start()
    {
        RegistrationServices registrationServices = new
RegistrationServices();
        comRegistrationToken =
registrationServices.RegisterTypeForComClients(typeof(SampleTask),
RegistrationClassContext.LocalServer,
RegistrationConnectionType.MultipleUse);

        // Either have the background task signal this handle when it
        // completes, or never signal this handle to keep this
        // process as the COM server until the process is closed.
        waitHandle.WaitOne();
    }

    public void Stop()
    {
        if (comRegistrationToken != 0)
        {
            RegistrationServices registrationServices = new
RegistrationServices();
registrationServices.UnregisterTypeForComClients(registrationCookie);
        }

        waitHandle.Set();
    }

    private int comRegistrationToken;
    private EventWaitHandle waitHandle;
}

var sampleTaskServer = new SampleTaskServer();
sampleTaskServer.Start();
```


Register the background task to run

1. Find out whether the background task is already registered by iterating through the [BackgroundTaskRegistration.AllTasks](#) property. *This step is important*; if your app doesn't check for existing background task registrations, it could easily register the task multiple times, causing issues with performance and maxing out the task's available CPU time before work can complete. An application is free to use the same entry point to handle all background tasks and use other properties like the [Name](#) or [TaskId](#) assigned to a [BackgroundTaskRegistration](#) to decide what work should be done.

The following example iterates on the [AllTasks](#) property and sets a flag variable to true if the task is already registered.

C#

```
var taskRegistered = false;
var sampleTaskName = "SampleTask";

foreach (var task in BackgroundTaskRegistration.AllTasks)
{
    if (task.Value.Name == sampleTaskName)
    {
        taskRegistered = true;
        break;
    }
}

// The code in the next step goes here.
```

1. If the background task is not already registered, use [BackgroundTaskBuilder](#) to create an instance of your background task. The task entry point should be the name of your background task class prefixed by the namespace.

The background task trigger controls when the background task will run. For a list of possible triggers, see the [Windows.ApplicationModel.Background](#) Namespace.

ⓘ Note

Only a subset of triggers are supported for packaged Win32 background tasks.

For example, this code creates a new background task and sets it to run it on a 15-minute recurring **TimeTrigger**:

C#

```
if (!taskRegistered)
{
    var builder = new BackgroundTaskBuilder();

    builder.Name = sampleTaskName;
    builder.SetTaskEntryPointClsid(typeof(SampleTask).GUID);
    builder.SetTrigger(new TimeTrigger(15, false));
}

// The code in the next step goes here.
```

1. You can add a condition to control when your task will run after the trigger event occurs (optional). For example, if you don't want the task to run until internet is available, use the condition **InternetAvailable**. For a list of possible conditions, see [SystemConditionType](#).

The following sample code assigns a condition requiring the user to be present:

C#

```
builder.AddCondition(new
SystemCondition(SystemConditionType.InternetAvailable));
// The code in the next step goes here.
```

4. Register the background task by calling the Register method on the **BackgroundTaskBuilder** object. Store the **BackgroundTaskRegistration** result so it can be used in the next step. Note that the register function may return errors in the form of exceptions. Be sure to call Register in a try-catch.

The following code registers the background task and stores the result:

C#

```
try
{
    var task = builder.Register();
}
catch (...)
{
    // Indicate an error was encountered.
```

```
}
```

Bringing it all together

The following code samples show the complete code required to run and register your COM Win32 background task:

▼ Complete Win32 app package manifest

XML

```
<?xml version="1.0" encoding="utf-8"?>
<Package
  xmlns="http://schemas.microsoft.com/appx/manifest/foundation/windows10"
  xmlns:uap="http://schemas.microsoft.com/appx/manifest/uap/windows10"

  xmlns:rescap="http://schemas.microsoft.com/appx/manifest/foundation/windows10/restrictedcapabilities"
  xmlns:com="http://schemas.microsoft.com/appx/manifest/com/windows10"
  IgnorableNamespaces="uap rescap com">

  <Identity
    Name="SamplePackagedWinMainBackgroundApp"
    Publisher="CN=Contoso"
    Version="1.0.0.0" />

  <Properties>
    <DisplayName>SamplePackagedWinMainBackgroundApp</DisplayName>
    <PublisherDisplayName>Contoso</PublisherDisplayName>
    <Logo>Images\StoreLogo.png</Logo>
  </Properties>

  <Dependencies>
    <TargetDeviceFamily Name="Windows.Desktop" MinVersion="10.0.19041.0"
    MaxVersionTested="10.0.19041.0" />
  </Dependencies>

  <Resources>
    <Resource Language="x-generate"/>
  </Resources>

  <Applications>
    <Application Id="App"
      Executable="SampleBackgroundApp\${targetnametoken$.exe"
      EntryPoint="${targetentrypoint$">

      <uap:VisualElements
        DisplayName="SampleBackgroundApp"
        Description="SampleBackgroundApp"
        BackgroundColor="transparent"
```

```

        Square150x150Logo="Images\Square150x150Logo.png"
        Square44x44Logo="Images\Square44x44Logo.png">
        <uap:DefaultTile Wide310x150Logo="Images\Wide310x150Logo.png" />
        <uap:SplashScreen Image="Images\SplashScreen.png" />
    </uap:VisualElements>

    <Extensions>
        <com:Extension Category="windows.comServer">
            <com:ComServer>
                <com:ExeServer
Executable="SampleBackgroundApp\SampleBackgroundApp.exe"
DisplayName="SampleBackgroundApp" Arguments="-StartSampleTaskServer">
                <com:Class Id="14C5882B-35D3-41BE-86B2-5106269B97E6"
DisplayName="Sample Task" />
                </com:ExeServer>
            </com:ComServer>
        </com:Extension>
    </Extensions>
</Application>
</Applications>

<Capabilities>
<rescap:Capability Name="runFullTrust" />
</Capabilities>
</Package>

```

▼ Complete background task code sample

C#

```

using System;
using System.IO; // Path
using System.Threading; // EventWaitHandle
using System.Collections.Generic; // Queue
using System.Runtime.InteropServices; // Guid, RegistrationServices
using Windows.ApplicationModel.Background; // IBackgroundTask

namespace PackagedWinMainBackgroundTaskSample
{
    // Background task implementation.
    // {14C5882B-35D3-41BE-86B2-5106269B97E6} is GUID to register this task
    with BackgroundTaskBuilder. Generate a random GUID before implementing.
    [ComVisible(true)]
    [ClassInterface(ClassInterfaceType.None)]
    [Guid("14C5882B-35D3-41BE-86B2-5106269B97E6")]
    [ComSourceInterfaces(typeof(IBackgroundTask))]
    public class SampleTask : IBackgroundTask
    {
        private volatile int cleanupTask; // flag used to indicate to Run
        method that it should exit
        private Queue<int> numbersQueue; // the data structure holding the

```

set of primes in memory

```
private const int maxPrimeNumber = 1000000000; // the number up to
which task will attempt to calculate primes
private const int queueDepthToWrite = 10; // how frequently this
task should flush its queue of primes
private const string numbersQueueFile = "numbersQueue.log"; // the
file to write to relative to AppData
```

```
public SampleTask()
{
    cleanupTask = 0;
    numbersQueue = new Queue<int>(queueDepthToWrite);
}

/// <summary>
/// This method writes all the numbers in the current queue to the
specified file.
```

```
/// </summary>
private void FlushNumbersToFile(Queue<int> queueToWrite)
{
```

```
    string logPath =
Path.Combine(ApplicationData.Current.LocalFolder.Path,

System.Diagnostics.Process.GetCurrentProcess().ProcessName);
```

```
    if (!Directory.Exists(logPath))
    {
        Directory.CreateDirectory(logPath);
    }
```

```
    logPath = Path.Combine(logPath, numbersQueueFile);
```

```
    const string delimiter = ", ";
    UnicodeEncoding unicodeEncoding = new UnicodeEncoding();
    // convert the queue to a list of comma separated values.
    string stringToWrite = String.Join(delimiter, queueToWrite);
    // Add the comma at the end.
    stringToWrite += delimiter;
```

```
    File.AppendAllText(logPath, stringToWrite);
}
```

```
/// <summary>
/// This method determines if the specified number is a prime
number.
```

```
/// </summary>
```

```
private bool IsPrimeNumber(int dividend)
```

```
{
    bool isPrime = true;
    for (int divisor = dividend - 1; divisor > 1; divisor -= 1)
    {
        if ((dividend % divisor) == 0)
        {
            isPrime = false;
        }
    }
}
```

```

        break;
    }
}

return isPrime;
}

/// <summary>
/// Given the current number, this method calculates the next prime
number (excluding the specified number).
/// </summary>
private int GetNextPrime(int previousNumber)
{
    int currentNumber = previousNumber + 1;
    while (!IsPrimeNumber(currentNumber))
    {
        currentNumber += 1;
    }

    return currentNumber;
}

/// <summary>
/// This method is the main entry point for the background task. The
system will believe this background task
/// is complete when this method returns.
/// </summary>
[MTAThread]
public void Run(IBackgroundTaskInstance taskInstance)
{
    // Start with the first applicable number.
    int currentNumber = 1;

    taskDeferral = taskInstance.GetDeferral();

    // Wire the cancellation handler.
    taskInstance.Canceled += this.OnCanceled;

    // Set the progress to indicate this task has started
    taskInstance.Progress = 10;

    // Calculate primes until a cancellation has been requested or
until
    // the maximum number is reached.
    while ((cleanupTask == 0) && (currentNumber < maxPrimeNumber)) {
        // Compute the next prime number and add it to our queue.
        currentNumber = GetNextPrime(currentNumber);
        numbersQueue.Enqueue(currentNumber);
        // Once the queue is filled to its max size, flush the
numbers to the file.
        if (numbersQueue.Count >= queueDepthToWrite)
        {
            FlushNumbersToFile(numbersQueue);
            numbersQueue.Clear();
        }
    }
}

```

```

    }

    // Flush any remaining numbers to the file as part of cleanup.
    FlushNumbersToFile(numbersQueue);

    if (taskDeferral != null)
    {
        taskDeferral.Complete();
    }
}

/// <summary>
/// This method is signaled when the system requests the background
task be canceled. This method will signal
/// to the Run method to clean up and return.
/// </summary>
[MTAThread]
public void OnCanceled(IBackgroundTaskInstance taskInstance,
BackgroundTaskCancellationReason cancellationReason)
{
    cleanupTask = 1;
}

}

// COM server startup code.
class SampleTaskServer
{
    SampleTaskServer()
    {
        comRegistrationToken = 0;
        waitHandle = new EventWaitHandle(false,
EventResetMode.AutoReset);
    }

    ~SampleTaskServer()
    {
        Stop();
    }

    public void Start()
    {
        RegistrationServices registrationServices = new
RegistrationServices();
        comRegistrationToken =
registrationServices.RegisterTypeForComClients(typeof(SampleTask),
RegistrationClassContext.LocalServer,
RegistrationConnectionType.MultipleUse);

        // Either have the background task signal this handle when it
completes, or never signal this handle to keep this
        // process as the COM server until the process is closed.
        waitHandle.WaitOne();
    }
}

```

```

        public void Stop()
        {
            if (comRegistrationToken != 0)
            {
                RegistrationServices registrationServices = new
RegistrationServices();

registrationServices.UnregisterTypeForComClients(registrationCookie);
            }

            waitHandle.Set();
        }

        private int comRegistrationToken;
        private EventWaitHandle waitHandle;
    }

    // Background task registration code.
    class SampleTaskRegistrar
    {
        public static void Register()
        {
            var taskRegistered = false;
            var sampleTaskName = "SampleTask";

            foreach (var task in BackgroundTaskRegistration.AllTasks)
            {
                if (task.Value.Name == sampleTaskName)
                {
                    taskRegistered = true;
                    break;
                }
            }

            if (!taskRegistered)
            {
                var builder = new BackgroundTaskBuilder();

                builder.Name = sampleTaskName;
                builder.SetTaskEntryPointClsid(typeof(SampleTask).GUID);
                builder.SetTrigger(new TimeTrigger(15, false));
            }

            try
            {
                var task = builder.Register();
            }
            catch (...)
            {
                // Indicate an error was encountered.
            }
        }
    }
}

```



```

// Application entry point.
static class Program
{
    [MTAThread]
    static void Main()
    {
        string[] commandLineArgs = Environment.GetCommandLineArgs();
        if (commandLineArgs.Length < 2)
        {
            // Open the WPF UI when no arguments are specified.
        }
        else
        {
            if (commandLineArgs.Contains("-RegisterSampleTask",
StringComparer.InvariantCultureIgnoreCase))
            {
                SampleTaskRegistrar.Register();
            }

            if (commandLineArgs.Contains("-StartSampleTaskServer",
StringComparer.InvariantCultureIgnoreCase))
            {
                var sampleTaskServer = new SampleTaskServer();
                sampleTaskServer.Start();
            }
        }

        return;
    }
}

```

Remarks

Unlike UWP apps that can run background tasks in modern standby, Win32 apps cannot run code from the lower power phases of modern standby. See [Modern Standby](#) to learn more.

[!NOTE] Download the [Win32 COM background task sample](#) to see similar code examples in the context of a complete Desktop Bridge app that uses background tasks.

See the following related topics for API reference, background task conceptual guidance, and more detailed instructions for writing apps that use background tasks.

Related topics

- [Respond to system events with background tasks](#)
- [Register a background task](#)
- [Set conditions for running a background task](#)
- [Use a maintenance trigger](#)
- [Handle a cancelled background task](#)
- [Monitor background task progress and completion](#)
- [Run a background task on a timer](#)
- [Create and register an in-process background task.](#)
- [Convert an out-of-process background task to an in-process background task](#)

Background task guidance

- [Guidelines for background tasks](#)
- [Debug a background task](#)
- [How to trigger suspend, resume, and background events in UWP apps \(when debugging\)](#)

Background Task API Reference

- [Windows.ApplicationModel.Background](#)

Port an out-of-process background task to an in-process background task

Article • 10/20/2022

The simplest way to port your out-of-process (OOP) background activity to in-process activity is to bring your [IBackgroundTask.Run](#) method code inside your application, and initiate it from [OnBackgroundActivated](#). The technique being described here is not about creating a shim from an OOP background task to an in-process background task; it's about rewriting (or porting) an OOP version to an in-process version.

If your app has multiple background tasks, the [Background Activation Sample](#) [↗](#) shows how you can use `BackgroundActivatedEventArgs.TaskInstance.Task.Name` to identify which task is being initiated.

If you are currently communicating between background and foreground processes, you can remove that state management and communication code.

Background tasks and trigger types that cannot be converted

- In-process background tasks don't support activating a VoIP background task.
- In-process background tasks don't support the following triggers:
[DeviceUseTrigger](#), [DeviceServicingTrigger](#) and [IoTStartupTask](#)

Debug a background task

Article • 10/20/2022

Important APIs

- [Windows.ApplicationModel.Background](#)

Learn how to debug a background task, including background task activation and debug tracing in the Windows event log.

Debugging out-of-process vs. in-process background tasks

This topic primarily addresses background tasks that run in a separate process than the host app. If you are debugging an in-process background task, then you won't have a separate background task project and can set a breakpoint on `OnBackgroundActivated()` (where your in-process background code runs) and see step 2 in [Trigger background tasks manually to debug background task code](#), below, for instructions about how to trigger your background code to execute.

Make sure the background task project is set up correctly

This topic assumes that you already have an existing app with a background task to debug. The following is specific to background tasks that run out-of-process and does not apply to in-process background tasks.

- In C# and C++, make sure the main project references the background task project. If this reference is not in place, the background task won't be included in the app package.
- In C# and C++, make sure the **Output type** of the background task project is "Windows Runtime Component".
- The background class must be declared in the entry point attribute in the package manifest.

Trigger background tasks manually to debug background task code

Background tasks can be triggered manually through Microsoft Visual Studio. Then you can step through the code and debug it.

1. In C#, put a breakpoint in the Run method of the background class (for in-process background tasks put the breakpoint in `App.OnBackgroundActivated()`), and/or write debugging output by using [System.Diagnostics](#).

In C++, put a breakpoint in the Run function of the background class (for in-process background tasks put the breakpoint in `App.OnBackgroundActivated()`), and/or write debugging output by using [OutputDebugString](#).

2. Run your application in the debugger and then trigger the background task using the **Lifecycle Events** toolbar. This drop down shows the names of the background tasks that can be activated by Visual Studio.

ⓘ Note

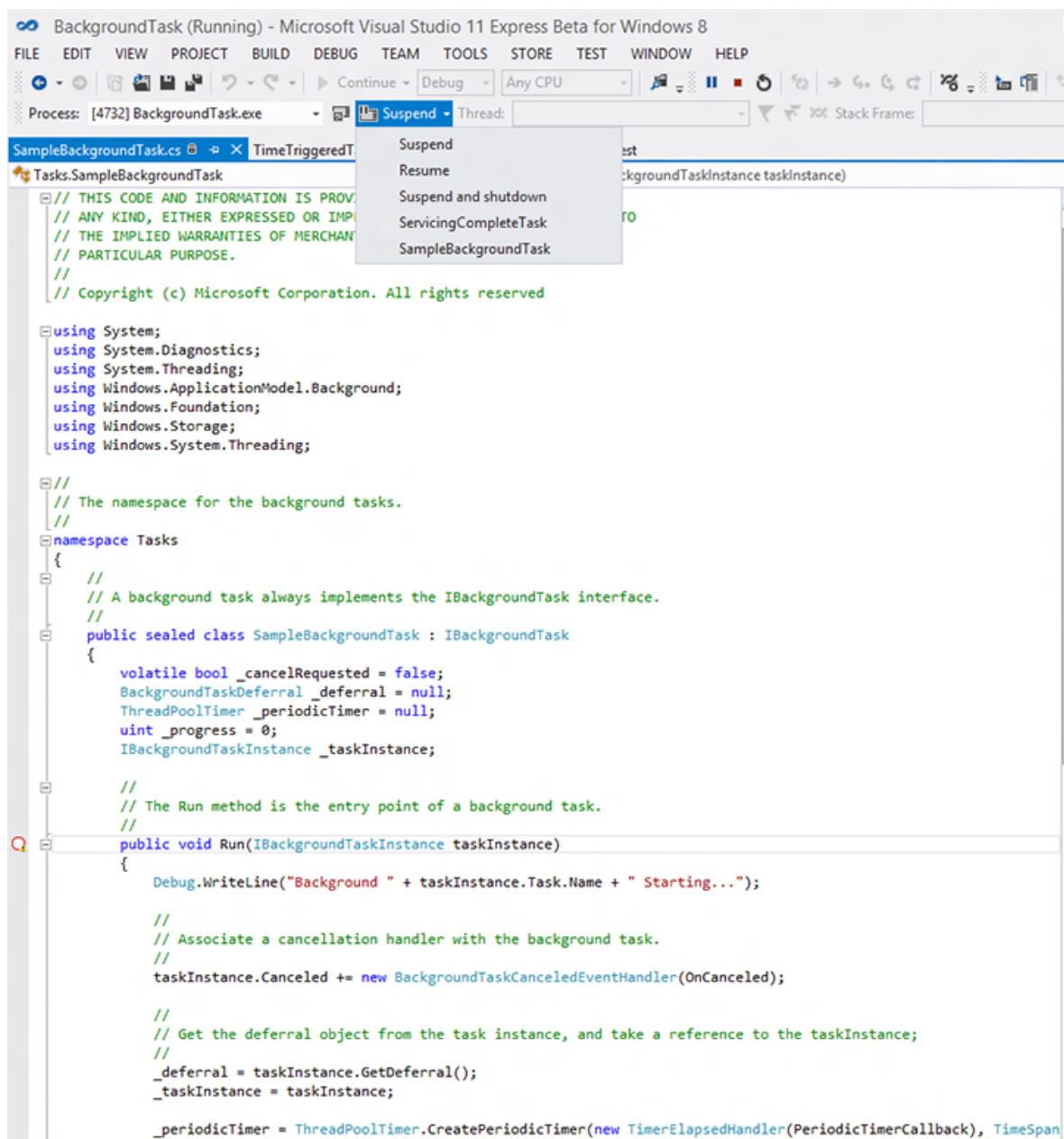
The Lifecycle Events toolbar options are not shown by default in Visual Studio. To show these options, right-click on the current toolbar in Visual Studio and ensure the option **Debug Location** is enabled.

For this to work, the background task must already be registered and it must still be waiting for the trigger. For example, if a background task was registered with a one-shot `TimeTrigger` and that trigger has already fired, launching the task through Visual Studio will have no effect.

ⓘ Note

Background tasks using the following triggers cannot be activated in this manner: **Application trigger**, **MediaProcessing trigger**, **ControlChannelTrigger**, **PushNotificationTrigger**, and background tasks using a **SystemTrigger** with the **SmsReceived** trigger type.

Application trigger and **MediaProcessingTrigger** can be signaled manually in code with `trigger.RequestAsync()`.



- When the background task activates, the debugger will attach to it and display debug output in VS.

Debug background task activation

ⓘ Note

This section is specific to background tasks the run out-of-process and does not apply to in-process background tasks.

Background task activation depends on three things:

- The name and namespace of the background task class
- The entry point attribute specified in the package manifest
- The entry point specified by your app when registering the background task

1. Use Visual Studio to note the entry point of the background task:

- In C# and C++, note the name and namespace of the background task class specified in the background task project.

2. Use the manifest designer to check that the background task is correctly declared in the package manifest:

- In C# and C++, the entry point attribute must match the background task namespace followed by the class name. For example:
RuntimeComponent1.MyBackgroundTask.
- All the trigger type(s) used with the task must also be specified.
- The executable MUST NOT be specified unless you are using the [ControlChannelTrigger](#) or [PushNotificationTrigger](#).

3. Windows only. To see the entry point used by Windows to activate the background task, enable debug tracing and use the Windows event log.

If you follow this procedure and the event log shows the wrong entry point or trigger for the background task, your app is not registering the background task correctly. For help with this task see [Register a background task](#).

a. Open event viewer by going to the Start screen and searching for eventvwr.exe.

b. Go to **Application and Services Logs** -> **Microsoft** -> **Windows** ->

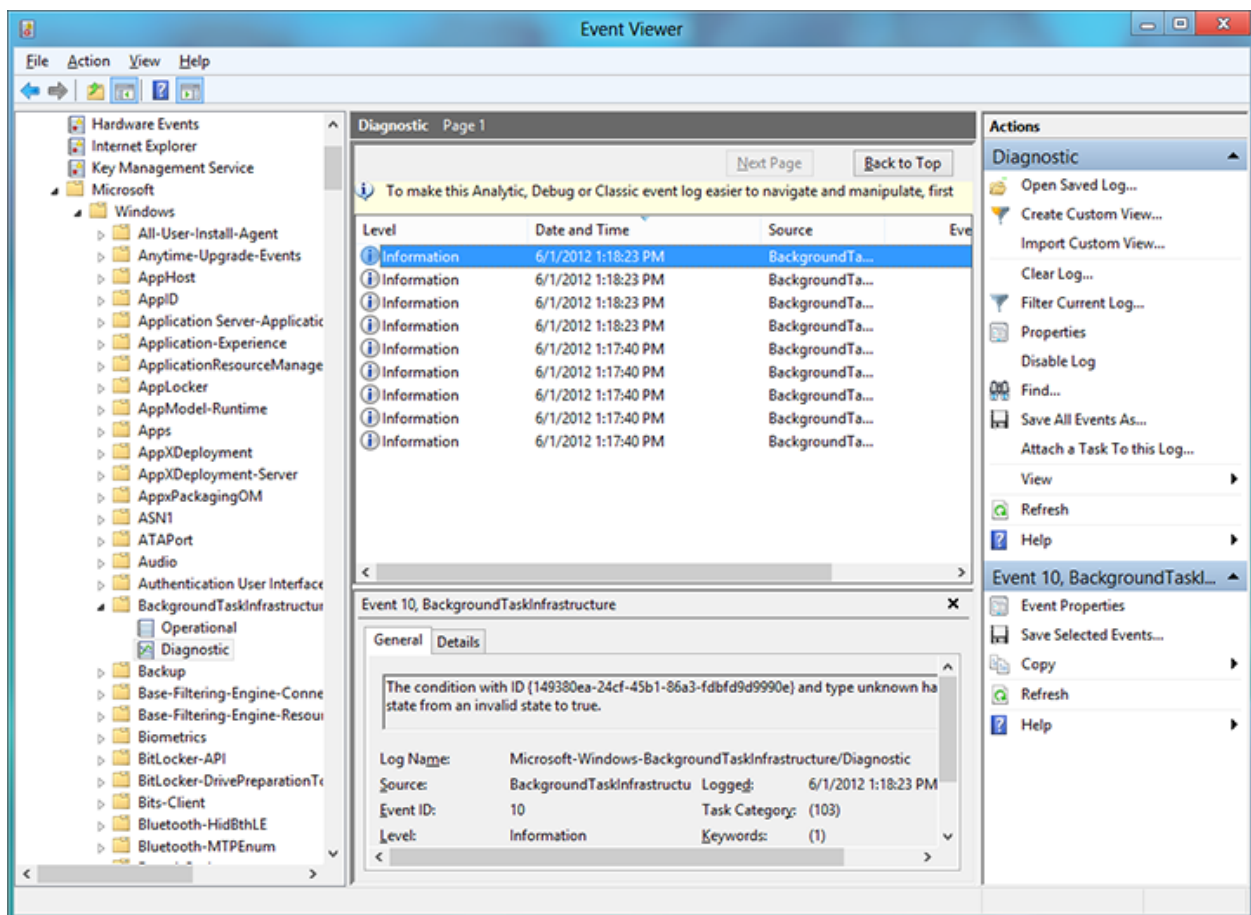
BackgroundTaskInfrastructure in the event viewer.

c. In the actions pane, select **View** -> **Show Analytic and Debug Logs** to enable the diagnostic logging.

d. Select the **Diagnostic log** and click **Enable Log**.

e. Now try using your app to register and activate the background task again.

f. View the diagnostic logs for detailed error information. This will include the entry point registered for the background task.



Background tasks and Visual Studio package deployment

If an app that uses background tasks is deployed using Visual Studio, and the version (major and/or minor) specified in Manifest Designer is then updated, subsequently re-deploying the app with Visual Studio may cause the app's background tasks to stall. This can be remedied as follows:

- Use Windows PowerShell to deploy the updated app (instead of Visual Studio) by running the script generated alongside the package.
- If you already deployed the app using Visual Studio and its background tasks are now stalled, reboot or log off/log in to get the app's background tasks working again.
- You can select the "Always re-install my package" debugging option to avoid this in C# projects.
- Wait until the app is ready for final deployment to increment the package version (don't change it while debugging).

Remarks

- Make sure your app checks for existing background task registrations before registering the background task again. Multiple registrations of the same background task can cause unexpected results by running the background task more than once each time it is triggered.
- If the background task requires lock screen access make sure to put the app on the lock screen before trying to debug the background task. For info on specifying manifest options for lock screen-capable apps, see [Declare background tasks in the application manifest](#).
- Background task registration parameters are validated at the time of registration. An error is returned if any of the registration parameters are invalid. Ensure that your app gracefully handles scenarios where background task registration fails - if instead your app depends on having a valid registration object after attempting to register a task, it may crash.

For more info on using VS to debug a background task see [How to trigger suspend, resume, and background events in UWP apps](#).

Related topics

- [Create and register an out-of-process background task](#)
- [Create and register an in-process background task](#)
- [Register a background task](#)
- [Declare background tasks in the application manifest](#)
- [Guidelines for background tasks](#)
- [How to trigger suspend, resume, and background events in UWP apps](#)
- [Overview of code analysis for .NET in Visual Studio](#)

Declare background tasks in the application manifest

Article • 10/27/2022

Important APIs

- [BackgroundTasks Schema](#)
- [Windows.ApplicationModel.Background](#)

Enable the use of background tasks by declaring them as extensions in the app manifest.

Important

This article is specific to out-of-process background tasks. In-process background tasks are not declared in the manifest.

Out-of-process background tasks must be declared in the app manifest or else your app will not be able to register them (an exception will be thrown). Additionally, out-of-process background tasks must be declared in the application manifest to pass certification.

This topic assumes you have created one or more background task classes, and that your app registers each background task to run in response to at least one trigger.

Add Extensions Manually

Open the application manifest (Package.appxmanifest) and go to the Application element. Create an Extensions element (if one doesn't already exist).

The following snippet is taken from the [background task sample](#) .

XML

```
<Application Id="App"
...
  <Extensions>
    <Extension Category="windows.backgroundTasks"
      EntryPoint="Tasks.SampleBackgroundTask">
      <BackgroundTasks>
        <Task Type="systemEvent" />
        <Task Type="timer" />
      </BackgroundTasks>
    </Extension>
  </Extensions>
</Application>
```

```

    </Extension>
    <Extension Category="windows.backgroundTasks"
EntryPoint="Tasks.ServicingComplete">
        <BackgroundTasks>
            <Task Type="systemEvent"/>
        </BackgroundTasks>
    </Extension>
</Extensions>
</Application>

```

Add a Background Task Extension

Declare your first background task.

Copy this code into the Extensions element (you will add attributes in the following steps).

XML

```

<Extensions>
    <Extension Category="windows.backgroundTasks" EntryPoint="">
        <BackgroundTasks>
            <Task Type="" />
        </BackgroundTasks>
    </Extension>
</Extensions>

```

1. Change the EntryPoint attribute to have the same string used by your code as the entry point when registering your background task (**namespace.classname**).

In this example, the entry point is

ExampleBackgroundTaskNameSpace.ExampleBackgroundTaskClassName:

XML

```

<Extensions>
    <Extension Category="windows.backgroundTasks"
EntryPoint="Tasks.ExampleBackgroundTaskClassName">
        <BackgroundTasks>
            <Task Type="" />
        </BackgroundTasks>
    </Extension>
</Extensions>

```

2. Change the list of Task Type attribute to indicate the type of task registration used with this background task. If the background task is registered with multiple trigger types, add additional Task elements and Type attributes for each one.

Note Make sure to list each of the trigger types you're using, or the background task will not register with the undeclared trigger types (the [Register](#) method will fail and throw an exception).

This snippet example indicates the use of system event triggers and push notifications:

XML

```
<Extension Category="windows.backgroundTasks"
EntryPoint="Tasks.BackgroundTaskClass">
  <BackgroundTasks>
    <Task Type="systemEvent" />
    <Task Type="pushNotification" />
  </BackgroundTasks>
</Extension>
```

Add multiple background task extensions

Repeat step 2 for each additional background task class registered by your app.

The following example is the complete Application element from the [background task sample](#). This shows the use of 2 background task classes with a total of 3 trigger types. Copy the Extensions section of this example, and modify it as needed, to declare background tasks in your application manifest.

XML

```
<Applications>
  <Application Id="App"
    Executable="$targetnametoken$.exe"
    EntryPoint="BackgroundTask.App">
    <uap:VisualElements
      DisplayName="BackgroundTask"
      Square150x150Logo="Assets\StoreLogo-sdk.png"
      Square44x44Logo="Assets\SmallTile-sdk.png"
      Description="BackgroundTask"

      BackgroundColor="#00b2f0">
      <uap:LockScreen Notification="badgeAndTileText"
BadgeLogo="Assets\smalltile-Windows-sdk.png" />
      <uap:SplashScreen Image="Assets\Splash-sdk.png" />
      <uap:DefaultTile DefaultSize="square150x150Logo"
Wide310x150Logo="Assets\tile-sdk.png" >
        <uap:ShowNameOnTiles>
          <uap:ShowOn Tile="square150x150Logo" />
          <uap:ShowOn Tile="wide310x150Logo" />
        </uap:ShowNameOnTiles>
      </uap:DefaultTile>
```

```

        </uap:VisualElements>

        <Extensions>
            <Extension Category="windows.backgroundTasks"
EntryPoint="Tasks.SampleBackgroundTask">
                <BackgroundTasks>
                    <Task Type="systemEvent" />
                    <Task Type="timer" />
                </BackgroundTasks>
            </Extension>
            <Extension Category="windows.backgroundTasks"
EntryPoint="Tasks.ServicingComplete">
                <BackgroundTasks>
                    <Task Type="systemEvent"/>
                </BackgroundTasks>
            </Extension>
        </Extensions>
    </Application>
</Applications>

```

Declare where your background task will run

You can specify where your background tasks run:

- By default, they run in the BackgroundTaskHost.exe process.
- In the same process as your foreground application.
- Use `ResourceGroup` to place multiple background tasks into the same hosting process, or to separate them into different processes.
- Use `SupportsMultipleInstances` to run the background process in a new process that gets its own resource limits (memory, cpu) each time a new trigger is fired.

Run in the same process as your foreground application

Here is example XML that declares a background task that runs in the same process as the foreground application.

XML

```

<Extensions>
    <Extension Category="windows.backgroundTasks"
EntryPoint="ExecModelTestBackgroundTasks.ApplicationTriggerTask">
        <BackgroundTasks>
            <Task Type="systemEvent" />
        </BackgroundTasks>
    </Extension>
</Extensions>

```

When you specify **EntryPoint**, your application receives a callback to the specified method when the trigger fires. If you do not specify an **EntryPoint**, your application receives the callback via [OnBackgroundActivated\(\)](#). See [Create and register an in-process background task](#) for details.

Specify where your background task runs with the ResourceGroup attribute.

Here is example XML that declares a background task that runs in a BackgroundTaskHost.exe process, but in a separate one than other instances of background tasks from the same app. Note the `ResourceGroup` attribute, which identifies which background tasks will run together.

XML

```
<Extensions>
  <Extension Category="windows.backgroundTasks"
EntryPoint="BackgroundTasks.SessionConnectedTriggerTask"
ResourceGroup="foo">
    <BackgroundTasks>
      <Task Type="systemEvent" />
    </BackgroundTasks>
  </Extension>
  <Extension Category="windows.backgroundTasks"
EntryPoint="BackgroundTasks.TimeZoneTriggerTask" ResourceGroup="foo">
    <BackgroundTasks>
      <Task Type="systemEvent" />
    </BackgroundTasks>
  </Extension>
  <Extension Category="windows.backgroundTasks"
EntryPoint="BackgroundTasks.TimerTriggerTask" ResourceGroup="bar">
    <BackgroundTasks>
      <Task Type="timer" />
    </BackgroundTasks>
  </Extension>
  <Extension Category="windows.backgroundTasks"
EntryPoint="BackgroundTasks.ApplicationTriggerTask" ResourceGroup="bar">
    <BackgroundTasks>
      <Task Type="general" />
    </BackgroundTasks>
  </Extension>
  <Extension Category="windows.backgroundTasks"
EntryPoint="BackgroundTasks.MaintenanceTriggerTask" ResourceGroup="foobar">
    <BackgroundTasks>
      <Task Type="general" />
    </BackgroundTasks>
  </Extension>
</Extensions>
```

Run in a new process each time a trigger fires with the SupportsMultipleInstances attribute

This example declares a background task that runs in a new process that gets its own resource limits (memory and CPU) every time a new trigger is fired. Note the use of `SupportsMultipleInstances` which enables this behavior. In order to use this attribute you must target SDK version '10.0.15063' (Windows 10 Creators Update) or later.

XML

```
<Package
  xmlns:uap4="http://schemas.microsoft.com/appx/manifest/uap/windows10/4"
  ...
  <Applications>
    <Application ...>
      ...
      <Extensions>
        <Extension Category="windows.backgroundTasks"
EntryPoint="BackgroundTasks.TimerTriggerTask">
          <BackgroundTasks uap4:SupportsMultipleInstances="true">
            <Task Type="timer" />
          </BackgroundTasks>
        </Extension>
      </Extensions>
    </Application>
  </Applications>
```

ⓘ Note

You cannot specify `ResourceGroup` or `ServerName` in conjunction with `SupportsMultipleInstances`.

Related topics

- [Debug a background task](#)
- [Register a background task](#)
- [Guidelines for background tasks](#)

Group background task registration

Article • 10/20/2022

Important APIs

[BackgroundTaskRegistrationGroup class](#)

Background tasks can now be registered in a group, which you can think of as a logical namespace. This isolation helps ensure that different components of an app, or different libraries, don't interfere with each other's background task registration.

When an app and the framework (or library) it uses registers a background task with the same name, the app could inadvertently remove the framework's background task registrations. App authors could also accidentally remove framework and library background task registrations because they could unregister all registered background tasks by using [BackgroundTaskRegistration.AllTasks](#). With groups, you can isolate your background task registrations so this doesn't happen.

Features of groups

- Groups can be uniquely identified by a GUID. They can also have an associated friendly name string which is easier to read while debugging.
- Multiple background tasks can be registered in a group.
- Background tasks registered in a group won't appear in [BackgroundTaskRegistration.AllTasks](#). Thus apps that currently use **BackgroundTaskRegistration.AllTasks** to unregister their tasks won't inadvertently unregister background tasks registered in a group. See [Unregister background tasks in a group](#) below to see how to unregister all background triggers that have been registered as part of a group.
- Each Background Task Registration will have a Group property to determine which group it is associated with.
- Registering In-Process background tasks with a group will cause the activation to go through [BackgroundTaskRegistrationGroup.BackgroundActivated](#) event instead of [Application.OnBackgroundActivated](#).

Register a background task in a group

The following shows how to register a background task (triggered by a time zone change, in this example) as part of a group.

C#

```
private const string groupFriendlyName = "myGroup";
private const string groupId = "3F2504E0-4F89-41D3-9A0C-0305E82C3301";
private const string myTaskName = "My Background Trigger";

public static void RegisterBackgroundTaskInGroup()
{
    BackgroundTaskRegistrationGroup group =
    BackgroundTaskRegistration.GetTaskGroup(groupId);
    bool isTaskRegistered = false;

    // See if this task already belongs to a group
    if (group != null)
    {
        foreach (var taskKeyValue in group.AllTasks)
        {
            if (taskKeyValue.Value.Name == myTaskName)
            {
                isTaskRegistered = true;
                break;
            }
        }
    }

    // If the background task is not in a group, register it
    if (!isTaskRegistered)
    {
        if (group == null)
        {
            group = new BackgroundTaskRegistrationGroup(groupId,
groupFriendlyName);
        }

        var builder = new BackgroundTaskBuilder();
        builder.Name = myTaskName;
        builder.TaskGroup = group; // we specify the group, here
        builder.SetTrigger(new
SystemTrigger(SystemTriggerType.TimeZoneChange, false));

        // Because builder.TaskEntryPoint is not specified,
        OnBackgroundActivated() will be raised when the background task is triggered
        BackgroundTaskRegistration task = builder.Register();
    }
}
```

Unregister background tasks in a group

The following shows how to unregister background tasks that were registered as part of a group. Because background tasks registered in a group don't appear in

[BackgroundTaskRegistration.AllTasks](#), you must iterate through the groups, find the background tasks registered to each group, and unregister them.

C#

```
private static void UnRegisterAllTasks()
{
    // Unregister tasks that are part of a group
    foreach (var groupKeyValue in BackgroundTaskRegistration.AllTaskGroups)
    {
        foreach (var groupedTask in groupKeyValue.Value.AllTasks)
        {
            groupedTask.Value.Unregister(true); // passing true to cancel
            currently running instances of this background task
        }
    }

    // Unregister tasks that aren't part of a group
    foreach(var taskKeyValue in BackgroundTaskRegistration.AllTasks)
    {
        taskKeyValue.Value.Unregister(true); // passing true to cancel
        currently running instances of this background task
    }
}
```

Register Persistent Events

When using Background Task Registration Groups with in-process background tasks, the background activations are directed towards the group's event instead of the one on the Application or CoreApplication object. This enables multiple components within your app to handle the activation rather than place all activation code paths in the Application object. The following shows how to register for the group's background activated event. First check [BackgroundTaskRegistration.GetTaskGroup](#) to determine if the group has already been registered. If not then create a new group with your id and friendly name. Then register an event handler to the BackgroundActivated event on the group.

C#

```
void RegisterPersistentEvent()
{
    var group = BackgroundTaskRegistration.GetTaskGroup(groupId);
    if (group == null)
    {
        group = new BackgroundTaskRegistrationGroup(groupId,
            groupFriendlyName);
    }
}
```

```
group.BackgroundActivated += MyEventHandler;  
}
```

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Handle a cancelled background task

Article • 10/20/2022

Important APIs

- [BackgroundTaskCanceledEventHandler](#)
- [IBackgroundTaskInstance](#)
- [ApplicationData.Current](#)

Learn how to make a background task that recognizes a cancellation request, stops work, and reports the cancellation to the app using persistent storage.

This topic assumes you have already created a background task class, including the **Run** method that is used as the background task entry point. To get started quickly building a background task, see [Create and register an out-of-process background task](#) or [Create and register an in-process background task](#). For more in-depth information on conditions and triggers, see [Support your app with background tasks](#).

This topic is also applicable to in-process background tasks. But instead of the **Run** method, substitute **OnBackgroundActivated**. In-process background tasks do not require you to use persistent storage to signal the cancellation because you can communicate the cancellation using app state since the background task is running in the same process as your foreground app.

Use the OnCanceled method to recognize cancellation requests

Write a method to handle the cancellation event.

ⓘ Note

For all device families except desktop, if the device becomes low on memory, background tasks may be terminated. If an out of memory exception is not surfaced, or the app doesn't handle it, then the background task will be terminated without warning and without raising the **OnCanceled** event. This helps to ensure the user experience of the app in the foreground. Your background task should be designed to handle this scenario.

Create a method named **OnCanceled** as follows. This method is the entry point called by the Windows Runtime when a cancellation request is made against your background

task.

C#

```
private void OnCanceled(
    IBackgroundTaskInstance sender,
    BackgroundTaskCancellationReason reason)
{
    // TODO: Add code to notify the background task that it is cancelled.
}
```

Add a flag variable called **_CancelRequested** to the background task class. This variable will be used to indicate when a cancellation request has been made.

C#

```
volatile bool _CancelRequested = false;
```

In the **OnCanceled** method you created in step 1, set the flag variable **_CancelRequested** to **true**.

The full [background task sample](#) **OnCanceled** method sets **_CancelRequested** to **true** and writes potentially useful debug output.

C#

```
private void OnCanceled(IBackgroundTaskInstance sender,
    BackgroundTaskCancellationReason reason)
{
    // Indicate that the background task is canceled.
    _cancelRequested = true;

    Debug.WriteLine("Background " + sender.Task.Name + " Cancel
    Requested...");
}
```

In the background task's **Run** method, register the **OnCanceled** event handler method before starting work. In an in-process background task, you might do this registration as part of your application initialization. For example, use the following line of code.

C#

```
taskInstance.Canceled += new BackgroundTaskCanceledEventHandler(OnCanceled);
```

Handle cancellation by exiting your background task

When a cancellation request is received, your method that does background work needs to stop work and exit by recognizing when `_cancelRequested` is set to `true`. For in-process background tasks, this means returning from the `OnBackgroundActivated` method. For out-of-process background tasks, this means returning from the `Run` method.

Modify the code of your background task class to check the flag variable while it's working. If `_cancelRequested` becomes set to true, stop work from continuing.

The [background task sample](#) includes a check that stops the periodic timer callback if the background task is canceled.

C#

```
if ((_cancelRequested == false) && (_progress < 100))
{
    _progress += 10;
    _taskInstance.Progress = _progress;
}
else
{
    _periodicTimer.Cancel();
    // TODO: Record whether the task completed or was cancelled.
}
```

ⓘ Note

The code sample shown above uses the `IBackgroundTaskInstance.Progress` property being used to record background task progress. Progress is reported back to the app using the `BackgroundTaskProgressEventArgs` class.

Modify the `Run` method so that after work has stopped, it records whether the task completed or was cancelled. This step applies to out-of-process background tasks because you need a way to communicate between processes when the background task was cancelled. For in-process background tasks, you can simply share state with the application to indicate the task was cancelled.

The [background task sample](#) records status in `LocalSettings`.

C#

```

if ((_cancelRequested == false) && (_progress < 100))
{
    _progress += 10;
    _taskInstance.Progress = _progress;
}
else
{
    _periodicTimer.Cancel();

    var settings = ApplicationData.Current.LocalSettings;
    var key = _taskInstance.Task.TaskId.ToString();

    // Write to LocalSettings to indicate that this background task ran.
    if (_cancelRequested)
    {
        settings.Values[key] = "Canceled";
    }
    else
    {
        settings.Values[key] = "Completed";
    }

    Debug.WriteLine("Background " + _taskInstance.Task.Name +
        (_cancelRequested ? " Canceled" : " Completed"));

    // Indicate that the background task has completed.
    _deferral.Complete();
}

```

Remarks

You can download the [background task sample](#) to see these code examples in the context of methods.

For illustrative purposes, the sample code shows only portions of the **Run** method (and callback timer) from the [background task sample](#).

Run method example

The complete **Run** method, and timer callback code, from the [background task sample](#) are shown below for context.

C#

```

// The Run method is the entry point of a background task.
public void Run(IBackgroundTaskInstance taskInstance)
{

```

```

        Debug.WriteLine("Background " + taskInstance.Task.Name + "
Starting...");

        // Query BackgroundWorkCost
        // Guidance: If BackgroundWorkCost is high, then perform only the
minimum amount
        // of work in the background task and return immediately.
        var cost = BackgroundWorkCost.CurrentBackgroundWorkCost;
        var settings = ApplicationData.Current.LocalSettings;
        settings.Values["BackgroundWorkCost"] = cost.ToString();

        // Associate a cancellation handler with the background task.
        taskInstance.Canceled += new
BackgroundTaskCanceledEventHandler(OnCanceled);

        // Get the deferral object from the task instance, and take a reference
to the taskInstance;
        _deferral = taskInstance.GetDeferral();
        _taskInstance = taskInstance;

        _periodicTimer = ThreadPoolTimer.CreatePeriodicTimer(new
TimerElapsedHandler(PeriodicTimerCallback), TimeSpan.FromSeconds(1));
    }

    // Simulate the background task activity.
    private void PeriodicTimerCallback(ThreadPoolTimer timer)
    {
        if ((_cancelRequested == false) && (_progress < 100))
        {
            _progress += 10;
            _taskInstance.Progress = _progress;
        }
        else
        {
            _periodicTimer.Cancel();

            var settings = ApplicationData.Current.LocalSettings;
            var key = _taskInstance.Task.Name;

            // Write to LocalSettings to indicate that this background task ran.
            settings.Values[key] = (_progress < 100) ? "Canceled with reason: "
+ _cancelReason.ToString() : "Completed";
            Debug.WriteLine("Background " + _taskInstance.Task.Name +
settings.Values[key]);

            // Indicate that the background task has completed.
            _deferral.Complete();
        }
    }
}

```

Related topics

- [Create and register an in-process background task.](#)
- [Create and register an out-of-process background task](#)
- [Declare background tasks in the application manifest](#)
- [Guidelines for background tasks](#)
- [Monitor background task progress and completion](#)
- [Register a background task](#)
- [Respond to system events with background tasks](#)
- [Run a background task on a timer](#)
- [Set conditions for running a background task](#)
- [Update a live tile from a background task](#)
- [Use a maintenance trigger](#)
- [Debug a background task](#)
- [How to trigger suspend, resume, and background events in UWP apps \(when debugging\)](#)

Monitor background task progress and completion

Article • 10/20/2022

Important APIs

- [BackgroundTaskRegistration](#)
- [BackgroundTaskProgressEventHandler](#)
- [BackgroundTaskCompletedEventHandler](#)

Learn how your app can recognize progress and completion reported by a background task that runs out-of-process. (For in-process background tasks, you can set shared variables to signify progress and completion.)

Background task progress and completion can be monitored by app code. To do so, the app subscribes to events from the background task(s) it has registered with the system.

- This topic assumes that you have an app that registers background tasks. To get started quickly building a background task, see [Create and register an in-process background task](#) or [Create and register an out-of-process background task](#). For more in-depth information on conditions and triggers, see [Support your app with background tasks](#).

Create an event handler to handle completed background tasks

Step 1

Create an event handler function to handle completed background tasks. This code needs to follow a specific footprint, which takes an [IBackgroundTaskRegistration](#) object and a [BackgroundTaskCompletedEventArgs](#) object.

Use the following footprint for the **OnCompleted** background task event handler method.

C#

```
private void OnCompleted(IBackgroundTaskRegistration task,
    BackgroundTaskCompletedEventArgs args)
{
```

```
// TODO: Add code that deals with background task completion.  
}
```

Step 2

Add code to the event handler that deals with the background task completion.

For example, the [background task sample](#) updates the UI.

C#

```
private void OnCompleted(IBackgroundTaskRegistration task,  
    BackgroundTaskCompletedEventArgs args)  
{  
    UpdateUI();  
}
```

Create an event handler function to handle background task progress

Step 1

Create an event handler function to handle completed background tasks. This code needs to follow a specific footprint, which takes in an [IBackgroundTaskRegistration](#) object and a [BackgroundTaskProgressEventArgs](#) object:

Use the following footprint for the OnProgress background task event handler method:

C#

```
private void OnProgress(IBackgroundTaskRegistration task,  
    BackgroundTaskProgressEventArgs args)  
{  
    // TODO: Add code that deals with background task progress.  
}
```

Step 2

Add code to the event handler that deals with the background task completion.

For example, the [background task sample](#) updates the UI with the progress status passed in via the *args* parameter:

C#

```
private void OnProgress(IBackgroundTaskRegistration task,
BackgroundTaskProgressEventArgs args)
{
    var progress = "Progress: " + args.Progress + "%";
    BackgroundTaskSample.SampleBackgroundTaskProgress = progress;
    UpdateUI();
}
```

Register the event handler functions with new and existing background tasks

Step 1

When the app registers a background task for the first time, it should register to receive progress and completion updates for it, in case the task runs while the app is still active in the foreground.

For example, the [background task sample](#)  calls the following function on each background task that it registers:

C#

```
private void AttachProgressAndCompletedHandlers(IBackgroundTaskRegistration
task)
{
    task.Progress += new BackgroundTaskProgressEventHandler(OnProgress);
    task.Completed += new BackgroundTaskCompletedEventHandler(OnCompleted);
}
```

Step 2

When the app launches, or navigates to a new page where background task status is relevant, it should get a list of background tasks currently registered and associate them with the progress and completion event handler functions. The list of background tasks currently registered by the application is kept in the [BackgroundTaskRegistration.AllTasks](#) property.

For example, the [background task sample](#)  uses the following code to attach event handlers when the SampleBackgroundTask page is navigated to:

C#

```
protected override void OnNavigatedTo(NavigationEventArgs e)
{
    foreach (var task in BackgroundTaskRegistration.AllTasks)
    {
        if (task.Value.Name ==
BackgroundTaskSample.SampleBackgroundTaskName)
        {
            AttachProgressAndCompletedHandlers(task.Value);

BackgroundTaskSample.UpdateBackgroundTaskStatus(BackgroundTaskSample.SampleB
ackgroundTaskName, true);
        }
    }

    UpdateUI();
}
```

Related topics

- [Create and register an in-process background task.](#)
- [Create and register an out-of-process background task](#)
- [Declare background tasks in the application manifest](#)
- [Handle a cancelled background task](#)
- [Register a background task](#)
- [Respond to system events with background tasks](#)
- [Set conditions for running a background task](#)
- [Update a live tile from a background task](#)
- [Use a maintenance trigger](#)
- [Run a background task on a timer](#)
- [Guidelines for background tasks](#)
- [Debug a background task](#)
- [How to trigger suspend, resume, and background events in UWP apps \(when debugging\)](#)

Register a background task

Article • 10/20/2022

Important APIs

- [BackgroundTaskRegistration class](#)
- [BackgroundTaskBuilder class](#)
- [SystemCondition class](#)

Learn how to create a function that can be re-used to safely register most background tasks.

This topic is applicable to both in-process background tasks and out-of-process background tasks. This topic assumes that you already have a background task that needs to be registered. (See [Create and register a background task that runs out-of-process](#) or [Create and register an in-process background task](#) for information about how to write a background task).

This topic walks through a utility function that registers background tasks. This utility function checks for existing registrations first before registering the task multiple times to avoid problems with multiple registrations, and it can apply a system condition to the background task. The walkthrough includes a complete, working example of this utility function.

Note

Universal Windows apps must call [RequestAccessAsync](#) before registering any of the background trigger types.

To ensure that your Universal Windows app continues to run properly after you release an update, you must call [RemoveAccess](#) and then call [RequestAccessAsync](#) when your app launches after being updated. For more information, see [Guidelines for background tasks](#).

Define the method signature and return type

This method takes in the task entry point, task name, a pre-constructed background task trigger, and (optionally) a [SystemCondition](#) for the background task. This method returns a [BackgroundTaskRegistration](#) object.

`taskEntryPoint` - for background tasks that run in out of process, this must be constructed as the namespace name, '.', and the name of the class containing your background class. The string is case-sensitive. For example, if you had a namespace "MyBackgroundTasks" and a class "BackgroundTask1" that contained your background class code, the string for `taskEntryPoint` would be "MyBackgroundTasks.BackgroundTask1". If your background task runs in the same process as your app (i.e., an in-process background task) `taskEntryPoint` should not be set.

C#

```
public static BackgroundTaskRegistration RegisterBackgroundTask(
    string taskEntryPoint,
    string name,
    IBackgroundTrigger trigger,
    IBackgroundCondition
condition)
{
    // We'll add code to this function in subsequent steps.
}
```

C++

```
BackgroundTaskRegistration^ MainPage::RegisterBackgroundTask(
    Platform::String ^
taskEntryPoint,
    Platform::String ^ taskName,
    IBackgroundTrigger ^ trigger,
    IBackgroundCondition ^
condition)
{
    // We'll add code to this function in subsequent steps.
}
```

Check for existing registrations

Check whether the task is already registered. It's important to check this because if a task is registered multiple times, it will run more than once whenever it's triggered; this can use excess CPU and may cause unexpected behavior.

You can check for existing registrations by querying the [BackgroundTaskRegistration.AllTasks](#) property and iterating on the result. Check the name of each instance – if it matches the name of the task you’re registering, then break out of the loop and set a flag variable so that your code can choose a different path in the next step.

Note Use background task names that are unique to your app. Ensure each background task has a unique name.

The following code registers a background task using the [SystemTrigger](#) we created in the last step:

C#

```
public static BackgroundTaskRegistration RegisterBackgroundTask(
    string taskEntryPoint,
    string name,
    IBackgroundTrigger trigger,
    IBackgroundCondition
condition)
{
    //
    // Check for existing registrations of this background task.
    //

    foreach (var cur in BackgroundTaskRegistration.AllTasks)
    {
        if (cur.Value.Name == name)
        {
            //
            // The task is already registered.
            //

            return (BackgroundTaskRegistration)(cur.Value);
        }
    }

    // We'll register the task in the next step.
}
```

C++

```
BackgroundTaskRegistration^ MainPage::RegisterBackgroundTask(
    Platform::String ^
taskEntryPoint,
    Platform::String ^ taskName,
    IBackgroundTrigger ^ trigger,
    IBackgroundCondition ^
```



```

condition)
{
    //
    // Check for existing registrations of this background task.
    //

    auto iter = BackgroundTaskRegistration::AllTasks->First();
    auto hascur = iter->HasCurrent;

    while (hascur)
    {
        auto cur = iter->Current->Value;

        if(cur->Name == name)
        {
            //
            // The task is registered.
            //

            return (BackgroundTaskRegistration ^)(cur);
        }

        hascur = iter->MoveNext();
    }

    // We'll register the task in the next step.
}

```

Register the background task (or return the existing registration)

Check to see if the task was found in the list of existing background task registrations. If so, return that instance of the task.

Then, register the task using a new [BackgroundTaskBuilder](#) object. This code should check whether the condition parameter is null, and if not, add the condition to the registration object. Return the [BackgroundTaskRegistration](#) returned by the [BackgroundTaskBuilder.Register](#) method.

Note Background task registration parameters are validated at the time of registration. An error is returned if any of the registration parameters are invalid. Ensure that your app gracefully handles scenarios where background task registration fails - if instead your app depends on having a valid registration object after attempting to register a task, it may crash. **Note** If you are registering a background task that runs in the same process as your app, send `String.Empty` or `null` for the `taskEntryPoint` parameter.

The following example either returns the existing task, or adds code that registers the background task (including the optional system condition if present):

C#

```
public static BackgroundTaskRegistration RegisterBackgroundTask(
    string taskEntryPoint,
    string name,
    IBackgroundTrigger trigger,
    IBackgroundCondition
condition)
{
    //
    // Check for existing registrations of this background task.
    //

    foreach (var cur in BackgroundTaskRegistration.AllTasks)
    {
        if (cur.Value.Name == taskName)
        {
            //
            // The task is already registered.
            //

            return (BackgroundTaskRegistration)(cur.Value);
        }
    }

    //
    // Register the background task.
    //

    var builder = new BackgroundTaskBuilder();

    builder.Name = name;

    // in-process background tasks don't set TaskEntryPoint
    if (taskEntryPoint != null && taskEntryPoint != String.Empty)
    {
        builder.TaskEntryPoint = taskEntryPoint;
    }
    builder.SetTrigger(trigger);

    if (condition != null)
    {
        builder.AddCondition(condition);
    }

    BackgroundTaskRegistration task = builder.Register();

    return task;
}
```

C++

```
BackgroundTaskRegistration^ MainPage::RegisterBackgroundTask(
    Platform::String ^
taskEntryPoint,
    Platform::String ^ taskName,
    IBackgroundTrigger ^ trigger,
    IBackgroundCondition ^
condition)
{
    //
    // Check for existing registrations of this background task.
    //

    auto iter = BackgroundTaskRegistration::AllTasks->First();
    auto hascur = iter->HasCurrent;

    while (hascur)
    {
        auto cur = iter->Current->Value;

        if(cur->Name == name)
        {
            //
            // The task is registered.
            //

            return (BackgroundTaskRegistration ^)(cur);
        }

        hascur = iter->MoveNext();
    }

    //
    // Register the background task.
    //

    auto builder = ref new BackgroundTaskBuilder();

    builder->Name = name;
    builder->TaskEntryPoint = taskEntryPoint;
    builder->SetTrigger(trigger);

    if (condition != nullptr) {
        builder->AddCondition(condition);
    }

    BackgroundTaskRegistration ^ task = builder->Register();

    return task;
}
```

Complete background task registration utility function

This example shows the completed background task registration function. This function can be used to register most background tasks, with the exception of networking background tasks.

C#

```
//
// Register a background task with the specified taskEntryPoint, name,
// trigger,
// and condition (optional).
//
// taskEntryPoint: Task entry point for the background task.
// taskName: A name for the background task.
// trigger: The trigger for the background task.
// condition: Optional parameter. A conditional event that must be true for
// the task to fire.
//
public static BackgroundTaskRegistration RegisterBackgroundTask(string
taskEntryPoint,
                                                                    string
taskName,

IBackgroundTrigger trigger,

IBackgroundCondition condition)
{
    //
    // Check for existing registrations of this background task.
    //

    foreach (var cur in BackgroundTaskRegistration.AllTasks)
    {
        if (cur.Value.Name == taskName)
        {
            //
            // The task is already registered.
            //

            return (BackgroundTaskRegistration)(cur.Value);
        }
    }

    //
    // Register the background task.
    //

    var builder = new BackgroundTaskBuilder();
```

```

        builder.Name = taskName;
        builder.TaskEntryPoint = taskEntryPoint;
        builder.SetTrigger(trigger);

        if (condition != null)
        {

            builder.AddCondition(condition);
        }

        BackgroundTaskRegistration task = builder.Register();

        return task;
    }

```

C++

```

//
// Register a background task with the specified taskEntryPoint, name,
// trigger,
// and condition (optional).
//
// taskEntryPoint: Task entry point for the background task.
// taskName: A name for the background task.
// trigger: The trigger for the background task.
// condition: Optional parameter. A conditional event that must be true for
// the task to fire.
//
BackgroundTaskRegistration^
MainPage::RegisterBackgroundTask(Platform::String ^ taskEntryPoint,

Platform::String ^ taskName,

IBackgroundTrigger ^ trigger,

IBackgroundCondition ^ condition)
{

    //
    // Check for existing registrations of this background task.
    //

    auto iter    = BackgroundTaskRegistration::AllTasks->First();
    auto hascur  = iter->HasCurrent;

    while (hascur)
    {
        auto cur = iter->Current->Value;

        if(cur->Name == name)
        {
            //
            // The task is registered.

```